boba_extension_dim01

Dimension 01: Boba Project Architecture Deep Dive

Executive Summary

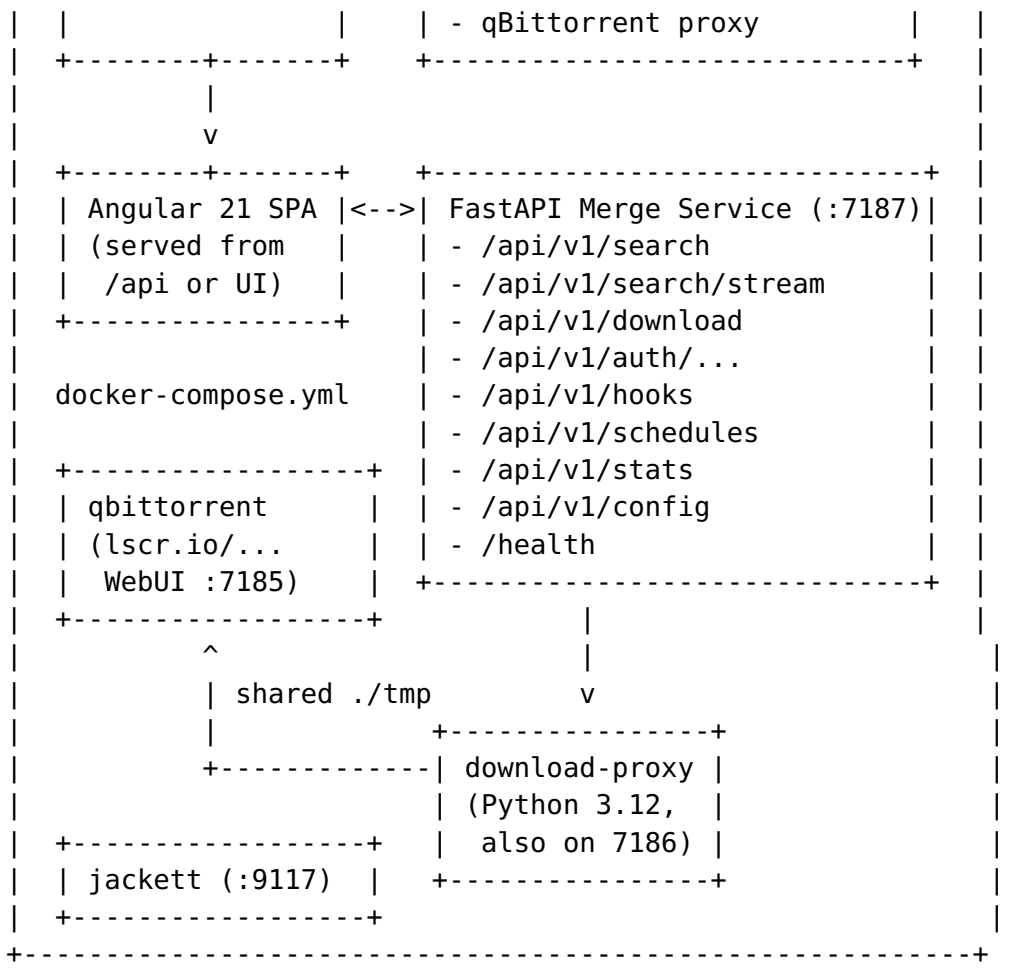
Boba-Base (GitHub: `milos85vasic/Boba-Base`) is a multi-tracker meta-search platform for qBittorrent, self-hosted and containerized. It consists of a FastAPI merge-search service, a Go/Gin backend (in parallel development), an Angular 21 SPA dashboard, a Python download-proxy, a private-tracker bridge (webui-bridge.py), Jackett integration, and a plugin system supporting 48+ torrent trackers. All runtime state is in-memory or JSON files – no relational database is used.

Key Finding for Browser Extension Integration: The FastAPI merge service on port 7187 exposes a full REST API (with OpenAPI/Swagger docs at `/docs`) including search, download, auth, hooks, scheduling, and SSE streaming. It serves the Angular SPA and supports CORS. The primary integration point for a browser extension is the `POST /api/v1/search` endpoint and `GET /api/v1/search/stream/{search_id}` SSE endpoint for real-time results, plus `POST /api/v1/download` to add torrents to qBittorrent.

---

1. Service Topology

```
+-----------------------------------------------------------+
|                     HOST MACHINE                          |
|  +----------------+    +-----------------------------+   |
|  | Browser (User) |    | webui-bridge.py (:7188)     |   |
|  |                |    | - Theme injection           |   |
|  |                |    | - Private tracker download  |   |
```




```
Optional (docker-compose.quality.yml):
  - SonarQube :9000   - Prometheus :9090   - Grafana :3000

New (Go backend, port 7187, profile: go):
  - qbittorrent-proxy-go (qBitTorrent-go/)
  - boba-jackett (:7189) - Jackett management API
```

Service Details

Service	Port	Container/ Process	Language	Role
qBittorrent	7185	`qbittorrent`	C++ (upstream)	Core torrent client, WebUI
Merge Search	7187	`qbittorrent-proxy`	Python 3.12 (FastAPI)	Main API, search orchestration, Angular hosting
Download Proxy	7186	same container	Python	

Service	Port	Container/ Process	Language	Role
				Legacy proxy, qBittorrent passthrough
WebUI Bridge	7188	Host process	Python 3	Private-tracker auth, theme injection
Jackett	9117	`jackett`	C#	External indexer aggregator
boba-jackett	7189	`boba-jackett`	Go (Gin)	Jackett management API
Go Proxy	7187	`qbittorrent-proxy-go`	Go (Gin)	Alternative merge search backend
Angular Dashboard	7187 (served)	static files	TypeScript/ Angular 21	SPA frontend

2. API Endpoints (Complete Catalog)

2.1 FastAPI Python Backend (Primary, Port 7187)

All endpoints served from the same FastAPI app in `download-proxy/src/api/__init__.py`.

Search Endpoints

Method	Path	Description	Request Body	Response
`POST`	`/api/v1/search`	Start async search (fire-and-forget)	`SearchRequest`	`SearchResponse` (status: running)
`POST`	`/api/v1/search/sync`	Blocking search (legacy)	`SearchRequest`	`SearchResponse` (full results)
`GET`			-	`SearchResponse`

Method	Path	Description	Request Body	Response
	`/api/v1/search/{search_id}`	Get search status + results		
GET	`/api/v1/search/stream/{search_id}`	**SSE stream** of real-time results	-	`text/event-stream`
POST	`/api/v1/search/{search_id}/abort`	Cancel running search	-	`{search_id, status}`

Download Endpoints

Method	Path	Description	Request Body	Response
POST	`/api/v1/download`	Add torrent(s) to qBittorrent	`DownloadRequest`	`{download_id, status, added_count, results}`
POST	`/api/v1/download/file`	Download .torrent file directly	`DownloadRequest`	`Blob` (application/x-bittorrent)
POST	`/api/v1/magnet`	Generate magnet link from URLs	`{result_id, download_urls}`	`{magnet, hashes}`
GET	`/api/v1/downloads/active`	List active qBittorrent downloads	-	`{downloads[], count}`

Authentication Endpoints

Method	Path	Description	Request Body	Response
POST	`/api/v1/auth/qbittorrent`	Login to qBittorrent WebUI	`{username, password, save?}`	`{status, version?, message?}`
GET	`/api/v1/auth/rutracker/status`	Check RuTracker session status	-	`{authenticated, status, message}`

Method	Path	Description	Request Body	Response
GET	/api/v1/auth/rutracker/captcha	Fetch CAPTCHA challenge	-	{captcha_required, captcha_image, captcha_token}
POST	/api/v1/auth/rutracker/login	Solve CAPTCHA + login	CaptchaLoginRequest	{authenticated, message}
POST	/api/v1/auth/rutracker/cookie-login	Login with browser cookies	CookieLoginRequest	{authenticated, message}
GET	/api/v1/auth/status	All trackers auth status	-	{trackers: {rutracker, kinozal, nnmclub, iptorrents, qbittorrent}}
POST	/api/v1/auth/qbittorrent/logout	Clear saved qBittorrent creds	-	{status}

Hook Endpoints

Method	Path	Description	Request Body	Response
GET	/api/v1/hooks	List all hooks	-	{hooks[], count}
POST	/api/v1/hooks	Create a new hook	HookCreateRequest	HookResponse
DELETE	/api/v1/hooks/{hook_id}	Delete a hook	-	{message, hook_id}
GET	/api/v1/hooks/logs	Get hook execution logs	?limit=50&hook_name=	{logs[], count}

Scheduler Endpoints

Method	Path	Description	Request Body	Response
GET	/api/v1/schedules	List scheduled searches	-	{schedules[], count}
POST	/api/v1/schedules	Create scheduled search	ScheduleCreateRequest	Schedule object
GET	/api/v1/schedules/{id}	Get scheduled search	-	Schedule object
PATCH	/api/v1/schedules/{id}	Update scheduled search	ScheduleUpdateRequest	{id, name, enabled}
DELETE	/api/v1/schedules/{id}	Delete scheduled search	-	{deleted, schedule_id}

Theme Endpoints

Method	Path	Description	Request Body	Response
GET	/api/v1/theme	Get current theme	-	Theme state object
PUT	/api/v1/theme	Update theme	{paletteId, mode}	Updated theme state
GET	/api/v1/theme/stream	SSE theme updates	-	text/event-stream

Utility Endpoints

Method	Path	Description	Response
GET	/health	Merge service health	{status, service, version}
GET			

Method	Path	Description	Response
	`/api/v1/bridge/health`	WebUI bridge health probe	`{healthy, status_code, bridge_url, port}`
GET	`/api/v1/config`	Service URLs for dashboard	`{qbittorrent_url, qbittorrent_internal_url, ...}`
GET	`/api/v1/stats`	Search stats + tracker list	`{active_searches, completed_searches, trackers[]}`
GET	`/docs`	**Swagger UI**	Interactive API docs
GET	`/openapi.json`	**OpenAPI 3.1 spec**	Machine-readable schema

2.2 Go/Gin Backend (Alternative, Port 7187, Profile: go)

The Go backend (`qBitTorrent-go/cmd/qbittorrent-proxy/main.go`) mirrors the Python API surface using the Gin framework. It is activated via `docker-compose.yml` profile `go`.

Key differences from Python backend: - Uses `qBittorrent` native search API (`/api/v2/search/start`) instead of direct plugin subprocess calls - SQLite database for system state (`boba.db` at `/config/boba.db`) - Admin/admin Basic Auth for mutating endpoints on boba-jackett (:7189)

Go boba-jackett API (port 7189):

Method	Path	Description
GET	`/healthz`	Health check
GET	`/openapi.json`	OpenAPI spec
GET/POST	`/api/v1/jackett/credentials`	Credential management
GET	`/api/v1/jackett/indexers`	List Jackett indexers
GET	`/api/v1/jackett/catalog`	Catalog of available trackers
POST	`/api/v1/jackett/catalog/refresh`	Refresh catalog
GET	`/api/v1/jackett/autoconfig/runs`	List autoconfig runs

Method	Path	Description
POST	/api/v1/jackett/autoconfig/run	Trigger autoconfig
GET/POST	/api/v1/jackett/overrides	Env variable overrides

2.3 qBittorrent WebUI API (Port 7185)

The native qBittorrent API is proxied through port 7186 (download-proxy) and 7188 (webui-bridge). Key endpoints used internally:

Method	Path	Description
POST	/api/v2/auth/login	Login (returns “Ok.”)
GET	/api/v2/app/version	Get qBittorrent version
POST	/api/v2/torrents/add	Add torrent (URLs or file upload)
GET	/api/v2/torrents/info	List all torrents
POST	/api/v2/search/start	Start qBittorrent search
GET	/api/v2/search/results	Get search results
POST	/api/v2/search/stop	Stop search
GET	/api/v2/search/plugins	List installed plugins

3. Request/Response Schemas

3.1 SearchRequest

```
{
  "query": "string (required, min_length=1)",
  "category": "string (default: 'all')",
  "limit": "integer (1-100, default: 50)",
  "enable_metadata": "boolean (default: true)",
  "validate_trackers": "boolean (default: true)",
  "sort_by": "string (default: 'seeds')",
  "sort_order": "string ('asc'|'desc', default: 'desc')"
}
```

3.2 SearchResponse

```
{
  "search_id": "uuid-string",
  "query": "original query",
```

```
  "status": "running|completed|failed|no_results|
        captcha_required",
  "results": ["SearchResultResponse"],
  "total_results": 0,
  "merged_results": 0,
  "trackers_searched": ["rutracker", "rutor", "1337x", ...],
  "errors": ["tracker: error message"],
  "tracker_stats": ["TrackerSearchStat"],
  "started_at": "ISO-8601 timestamp",
  "completed_at": "ISO-8601 timestamp|null"
}
```

3.3 SearchResultResponse

```
{
  "name": "torrent name",
  "size": "human readable or bytes",
  "seeds": 42,
  "leechers": 7,
  "download_urls": ["https://..."],
  "quality": "uhd_4k|full_hd|hd|sd|unknown",
  "content_type": "movie|tv|anime|music|game|software|ebook|
        other",
  "desc_link": "description URL",
  "tracker": "tracker name",
  "sources": [{"tracker": "...", "seeds": 42, "leechers": 7}],
  "metadata": {
    "source": "OMDb|TMDB",
    "title": "canonical title",
    "year": 2024,
    "poster_url": "https://...",
    "overview": "plot description",
    "genres": ["Action", "Sci-Fi"]
  },
  "freeleech": false
}
```

3.4 DownloadRequest

```
{
  "result_id": "merged result id (uuid)",
  "download_urls": ["https://tracker/download/...", "magnet:?
        xt=urn:btih:..."]
}
```

3.5 TrackerSearchStat

```
{
  "name": "tracker identifier",
  "tracker_url": "base URL",
```

```
  "status": "pending|running|success|empty|error|timeout|
        cancelled",
  "results_count": 42,
  "started_at": "ISO-8601|null",
  "completed_at": "ISO-8601|null",
  "duration_ms": 1234,
  "error": "error message|null",
  "error_type": "upstream_http_403|dns_failure|plugin_crashed|
        null",
  "authenticated": true,
  "attempt": 1,
  "http_status": 200,
  "category": "all",
  "query": "original query",
  "notes": {}
}
```

---

4. Authentication Mechanisms

4.1 qBittorrent WebUI Auth

- **Default credentials**: `admin` / `admin` (configurable via `.env`)
- **Login flow**: `POST /api/v2/auth/login` with form data `username` + `password`
- **Session**: Cookie-based (`SID` cookie returned)
- **Saved credentials**: Stored at `/config/download-proxy/qbittorrent_creds.json` (optional)
- **Endpoint**: `POST /api/v1/auth/qbittorrent` (proxy through merge service)

4.2 Private Tracker Auth

RuTracker: - Username/password from env (`RUTRACKER_USERNAME`, `RUTRACKER_PASSWORD`) - CAPTCHA challenge flow: `GET /api/v1/auth/rutracker/captcha` -> solve -> `POST /api/v1/auth/rutracker/login` - Cookie-based session stored in `SearchOrchestrator._tracker_sessions` - Alternative: Cookie login via `POST /api/v1/auth/rutracker/cookie-login`

Kinozal: - Username/password from env (`KINOZAL_USERNAME`, `KINOZAL_PASSWORD`) - Falls back to IPTorrents credentials if not set - Session-based authentication

NNM-Club: - Cookie-based only (`NNMCLUB_COOKIES`) - Very sensitive to proxies - Session stored in orchestrator

IPTorrents: - Username/password from env (`IPTORRENTS_USERNAME`, `IPTORRENTS_PASSWORD`) - Freeleech-only mode enforced - Session-based authentication

4.3 Jackett API Auth

- API key from env (`JACKETT_API_KEY`)
- Passed as query parameter or header to Jackett on port 9117

4.4 boba-jackett (Go) Auth

- **Admin Basic Auth**: `admin` / `admin` hardcoded
- GET/HEAD/OPTIONS pass through unauthenticated
- Mutating endpoints (POST/PATCH/DELETE) require `Authorization: Basic YWRtaW46YWRtaW4=`

4.5 CORS

- Configured via `ALLOWED_ORIGINS` env var (default: `*`)
- All origins allowed in development

---

5. Data Models

5.1 Internal Dataclasses (Python, `merge_service/search.py`)

SearchMetadata – The core search tracking object:

```python
@dataclass
class SearchMetadata:
    search_id: str          # UUID
    query: str
    category: str = "all"
    started_at: datetime    # UTC
    completed_at: datetime | None
    total_results: int = 0
    merged_results: int = 0
    trackers_searched: list[str]
    errors: list[str]
    status: str = "running"  # running|completed|failed|aborted
    tracker_stats: dict[str, TrackerSearchStat]
```

SearchResult – Individual torrent result from a tracker:

```python
@dataclass
class SearchResult:
    name: str
    link: str               # Download URL or magnet
    size: str               # Human-readable
    seeds: int
    leechers: int
    engine_url: str
    desc_link: str | None
    pub_date: str | None
    tracker: str | None
    category: str | None
    freeleech: bool = False
    content_type: str | None
    quality: str | None
```

MergedResult – Deduplicated result combining multiple sources:

```python
@dataclass
class MergedResult:
    canonical_identity: CanonicalIdentity
    original_results: list[SearchResult]
    total_seeds: int
    total_leechers: int
    best_quality: QualityTier | None
    download_urls: list[str]
    created_at: datetime
```

CanonicalIdentity – Deduplication fingerprint:

```python
@dataclass
class CanonicalIdentity:
    infohash: str | None
    title: str | None
    year: int | None
    content_type: ContentType | None   # Enum:
        movie,tv,anime,music,...
    season: int | None
    episode: int | None
    resolution: str | None
    codec: str | None
    group: str | None
    metadata_source: str | None
```

ContentType Enum: `movie`, `tv`, `anime`, `music`, `audiobook`, `game`, `software`, `ebook`, `other`, `unknown`

QualityTier Enum: `sd`, `hd`, `full_hd`, `uhd_4k`, `uhd_8k`, `unknown`

5.2 JSON Persistence (No Relational DB)

File	Owner	Contents
`/config/download-proxy/hooks.json`	`api/hooks.py`	Array of `HookConfig` dicts
`/config/download-proxy/qbittorrent_creds.json`	`api/routes.py`	`{username, password}`
`/config/merge-service/scheduling.json`	`merge_service/scheduler.py`	Array of `ScheduledSearch` dicts
`/config/merge-service/theme.json`	`api/theme_state.py`	Theme palette + mode
`/config/boba.db`	Go backend only	SQLite (credentials, indexers, catalog, runs, overrides)

5.3 Go Models (`qBitTorrent-go/internal/models/`)

Key Go structs mirror the Python dataclasses: - `SearchRequest` / `SearchResponse` / `TrackerStat` - `DownloadRequest` / `DownloadResult` / `URLDownloadResult` - `QBittorrentAuthRequest` / `QBittorrentAuthResponse` - `Hook` / `Schedule` / `ThemeState`

---

6. Plugin Architecture

6.1 Plugin System Overview

Boba uses the **qBittorrent nova3 plugin format** – Python classes implementing a standard interface. Plugins are installed to `/config/qBittorrent/nova3/engines/`.

Plugin execution flow: 1. SearchOrchestrator fans out to each enabled tracker 2. For public trackers: `subprocess python3 -c <engine>.search()` (isolated process) 3. For private trackers: Direct aiohttp calls with session cookies (Python) or via qBittorrent search API (Go) 4. Results parsed from stdout JSON (public) or HTML (private) 5. Deduplicator merges by infohash > title+size > fuzzy match

6.2 Plugin Interface

Each plugin implements:

```
class MyTracker:
    url = 'https://tracker.example.com'
    name = 'MyTracker'
    supported_categories = {'all': True, 'movies': '14', 'tv':
        '15', ...}

    def __init__(self):
        pass

    def search(self, what, cat='all'):
        """Return list of dicts: {link, name, size, seeds,
        leechers, engine_url, desc_link}"""
        pass

    def download_torrent(self, url):
        """Download torrent file, return bytes or file path"""
        pass
```

6.3 Plugin Categories

48 built-in plugins organized into: - **Core plugins** (`plugins/`): rutracker, rutor, kinozal, nnmclub, iptorrents, 1337x, piratebay, yts, eztv, limetorrents, torlock, torrentgalaxy, solidtorrents, nyaa, kickass, megapeer, tokyotoshokan, bitsearch, gamestorrents, audiobookbay, rockbox, glotorrents, snowfl, bt4g, extratorrent, therarbg, etc. - **Community plugins** (`plugins/community/`): academic, ali213, anilibra, btsow, linuxtracker, pctorrent, pirateiro, torrentscsv, xfsub, yihua, yourbittorrent, etc. - **WebUI-compatible plugins** (`plugins/webui_compatible/`): kinozal, nnmclub, rutracker (specialized for WebUI bridge)

6.4 Public Tracker Registry (Python)

```
PUBLIC_TRACKERS = {
    "1337x": "https://1337x.to",
    "piratebay": "https://thepiratebay.org",
    "yts": "https://yts.lt",
    "rutor": "https://rutor.info/",
    "limetorrents": "https://www.limetorrents.lol",
    "torrentgalaxy": "https://torrentgalaxy.one",
    "nyaa": "https://nyaa.si",
    # ... 40+ more
}
```

Dead trackers (excluded by default, enable with `ENABLE_DEAD_TRACKERS=1`): `ali213`, `audiobookbay`, `bitru`, `bt4g`, `btsow`, `extratorrent`, `eztv`, `one337x`, `pctorrent`, `solidtorrents`, `therarbg`, `torrentfunk`, `xfsub`, `yihua`

6.5 Private Tracker Registry

```
PRIVATE_TRACKERS = {
    "rutracker": "https://rutracker.org",
    "kinozal": "https://kinozal.tv",
    "nnmclub": "https://nnm-club.me",
    "iptorrents": "https://iptorrents.com",
}
```

---

7. Search Flow (Detailed)

7.1 Search Lifecycle

```
1. Client -> POST /api/v1/search {query, category}
2. FastAPI -> SearchOrchestrator.start_search()
3. SearchOrchestrator creates SearchMetadata with UUID
4. SearchOrchestrator._run_search() fires in background
asyncio.Task
5. For each enabled tracker:
   a. TrackerSearchStat created with status "pending"
   b. Subprocess or aiohttp call initiated
   c. Stat flips to "running"
   d. Results collected into _tracker_results[search_id]
[tracker_name]
   e. Stat flips to "success", "empty", "error", or "timeout"
6. Deduplicator.merge_results() combines by CanonicalIdentity
7. MetadataEnricher fills poster, overview, genres from OMDB/TMDB/
AniList
8. Status flips to "completed"
9. Client polls GET /search/{id} or receives SSE events
```

7.2 SSE Event Types

Event	Payload	When
`search_start`	`{search_id, status: "started"}`	Stream begins
`tracker_started`	`TrackerSearchStat`	Tracker status pending -> running

Event	Payload	When
`tracker_completed`	`TrackerSearchStat`	Tracker reaches terminal state
`result_found`	`{search_id, name, seeds, leechers, tracker, size, link, content_type, quality}`	New result discovered
`results_update`	`{search_id, total_results, merged_results, trackers_searched}`	Result count changes
`search_complete`	`SearchMetadata.to_dict()`	All trackers finished
`error`	`{error, search_id}`	Search not found or error
`close`	`{search_id, reason}`	Client disconnects

7.3 Deduplication Strategy

```
Tier 1: Infohash match (exact, strongest)
Tier 2: Normalized title + size match
Tier 3: Levenshtein fuzzy title match
Tier 4: Weak heuristic fallback
```

IPTorrents freeleech results are **never merged** with non-freeleech sources (to preserve ratio).

---

8. Download Flow (Detailed)

8.1 Public/Magnet Downloads

```
1. Client -> POST /api/v1/download {result_id, download_urls}
2. API authenticates with qBittorrent via saved or env credentials
3. For each URL:
   a. If tracker URL: fetch .torrent via
orchestrator.fetch_torrent()
   b. If magnet link: POST to /api/v2/torrents/add with urls param
   c. If direct URL: POST to /api/v2/torrents/add with urls param
4. Return {download_id, status, added_count, results}
```

8.2 Private Tracker Downloads (webui-bridge.py)

```
1. Download request hits :7188 (webui-bridge)
2. Bridge identifies tracker from URL patterns
3. Bridge calls nova2dl.py <engine> <url>
4. nova2dl imports plugin, calls download_torrent()
5. Plugin authenticates with tracker (session/cookies)
6. Plugin downloads .torrent to /shared-tmp/<uuid>.torrent
7. Bridge uploads file to qBittorrent via multipart POST
8. Bridge returns OK to client
```

---

9. Environment Variables (Complete Reference)

9.1 Required Variables

Variable	Description	Default
`RUTRACKER_USERNAME`	RuTracker login	(none)
`RUTRACKER_PASSWORD`	RuTracker password	(none)
`KINOZAL_USERNAME`	Kinozal login	(none)
`KINOZAL_PASSWORD`	Kinozal password	(none)
`NNMCLUB_USERNAME`	NNM-Club username	(none)
`NNMCLUB_COOKIES`	NNM-Club browser cookies	(none)
`IPTORRENTS_USERNAME`	IPTorrents login	(none)
`IPTORRENTS_PASSWORD`	IPTorrents password	(none)

9.2 Service Configuration

Variable	Description	Default
`QBITTORRENT_HOST`	qBittorrent hostname	`localhost`
`QBITTORRENT_PORT`	qBittorrent WebUI port	`7185`
`QBITTORRENT_URL`	Full qBittorrent URL	`http://localhost:7185`
`QBITTORRENT_USER` / `QBITTORRENT_USERNAME`	WebUI username	`admin`
`QBITTORRENT_PASS` / `QBITTORRENT_PASSWORD`	WebUI password	`admin`
`WEBUI_PORT`	Internal WebUI port	`7185`

Variable	Description	Default
WEBUI_USERNAME	WebUI username (container)	admin
WEBUI_PASSWORD	WebUI password (container)	admin
PROXY_PORT	Download proxy port	7186
MERGE_SERVICE_PORT	FastAPI port	7187
BRIDGE_PORT	WebUI bridge port	7188
MERGE_SERVICE_HOST	FastAPI bind host	0.0.0.0
SERVER_PORT	Go server port	7187

9.3 Performance Tuning

Variable	Description	Default
MAX_CONCURRENT_SEARCHES	Max parallel searches	5
MAX_CONCURRENT_TRACKERS	Max parallel tracker calls	10
MAX_CONCURRENT_SSE_STREAMS	Max open SSE connections	32
PUBLIC_TRACKER_DEADLINE_SECONDS	Per-tracker timeout	15
PLUGIN_TIMEOUT	Plugin execution timeout	10
SSE_TIMEOUT	SSE connection timeout	30

9.4 Feature Toggles

Variable	Description	Default
ENABLE_DEAD_TRACKERS	Include dead trackers	0
DISABLE_THEME_INJECTION	Disable theme	0
JACKETT_API_KEY	Jackett API key	(none)
JACKETT_URL	Jackett URL	http://localhost:9117
ALLOWED_ORIGINS	CORS origins	*
LOG_LEVEL	Logging level	INFO

9.5 Metadata Enrichment

Variable	Description
`OMDB_API_KEY`	OMDb API key
`TMDB_API_KEY`	TMDB API key
`ANILIST_CLIENT_ID`	AniList client ID
`TVDB_API_KEY`	TVDB API key

9.6 RuTracker Specific

Variable	Description	Default
`RUTRACKER_MIRRORS`	Comma-separated mirror URLs	`https://rutracker.org,...`
`PENDING_CAPTCHAS_MAX`	Max cached CAPTCHA challenges	`1024`
`PENDING_CAPTCHAS_TTL_SECONDS`	CAPTCHA challenge TTL	`900`

10. Frontend Architecture (Angular 21)

10.1 App Structure

```
frontend/src/app/
  app.ts                    # Root component (standalone)
  app.config.ts             # App configuration
  app.routes.ts             # Route definitions
  components/
    dashboard/              # Main search dashboard
    qbit-login-dialog/      # qBittorrent login modal
    confirm-dialog/         # Confirmation dialogs
    magnet-dialog/          # Magnet link generator
    theme-picker/           # Theme selection
    toast-container/        # Toast notifications
    tracker-stat-dialog/    # Tracker status detail
    site-footer/            # Footer
  services/
    api.service.ts          # HTTP client for all API calls
    sse.service.ts          # Server-Sent Events handler
    theme.service.ts        # Theme state management
```

```
    dialog.service.ts       # Dialog management
    toast.service.ts        # Toast notifications
  models/
    search.model.ts         # TypeScript interfaces
    palette.model.ts        # Theme palette definitions
  jackett/                  # Jackett management UI
    credentials/
    indexers/
    jackett.routes.ts
```

10.2 API Service Pattern

The frontend uses Angular's `HttpClient` with a base URL of `''` (same origin, since the SPA is served from the FastAPI app on :7187):

```typescript
// api.service.ts
private baseUrl = '';  // Same-origin requests

search(req: SearchRequest): Observable<SearchResponse> {
  return this.http.post<SearchResponse>(`${this.baseUrl}/api/v1/
        search`, req);
}

// SSE connection
connect(searchId: string): void {
  this.eventSource = new EventSource(`/api/v1/search/stream/$
        {searchId}`);
  // Listens for: search_start, result_found, results_update,
        search_complete
}
```

10.3 Dashboard Features

- **Search form**: Query input, category dropdown, sort controls
- **Live results table**: Real-time updates via SSE
- **Tracker status chips**: Per-tracker progress indicators
- **CAPTCHA modal**: RuTracker CAPTCHA challenge solving
- **Download buttons**: Direct download, magnet link, file download
- **Theme picker**: 8 palettes, dark/light mode
- **qBittorrent login**: Modal for WebUI authentication

---

11. Hook System

11.1 Hook Events

Event	When Fired
`search_start`	Search initiated
`search_progress`	Intermediate results available
`search_complete`	All trackers finished
`download_start`	Download request received
`download_progress`	Download in progress
`download_complete`	All URLs processed
`merge_complete`	Deduplication finished
`validation_complete`	Tracker validation finished

11.2 Hook Registration

```
curl -X POST http://localhost:7187/api/v1/hooks \
  -H "Content-Type: application/json" \
  -d '{
    "name": "notify_on_download",
    "event": "download_complete",
    "script_path": "/config/scripts/notify.sh",
    "enabled": true,
    "timeout": 30,
    "environment": {"WEBHOOK_URL": "https://hooks.example.com/
        notify"}
  }'
```

---

12. Browser Extension Integration Points

12.1 Primary Integration Strategy

A browser extension should communicate directly with the **FastAPI merge service on port 7187**. The extension can:

1. **Search**: `POST /api/v1/search` + `GET /api/v1/search/stream/{id}` (SSE)
2. **Download**: `POST /api/v1/download` to add to qBittorrent
3. **Get torrent file**: `POST /api/v1/download/file`
4. **Generate magnet**: `POST /api/v1/magnet`

5. **Check auth**: `GET /api/v1/auth/status`
6. **Get config**: `GET /api/v1/config` (for qBittorrent URL)

12.2 Extension API Usage Examples

```
// 1. Start a search
const searchResponse = await fetch('http://localhost:7187/api/v1/
        search', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ query: 'Ubuntu 24.04', category: 'all',
        limit: 50 })
});
const { search_id } = await searchResponse.json();

// 2. Stream results via SSE
const eventSource = new EventSource(
  `http://localhost:7187/api/v1/search/stream/${search_id}`
);
eventSource.addEventListener('result_found', (e) => {
  const result = JSON.parse(e.data);
  console.log(`Found: ${result.name} (${result.seeds} seeds)`);
});
eventSource.addEventListener('search_complete', () => {
  eventSource.close();
});

// 3. Download a torrent
await fetch('http://localhost:7187/api/v1/download', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    result_id: 'uuid-of-result',
    download_urls: ['https://tracker.example.com/download/12345']
  })
});

// 4. Get torrent file as blob
const blobResponse = await fetch('http://localhost:7187/api/v1/
        download/file', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ result_id: 'uuid', download_urls:
        ['https://...'] })
});
const blob = await blobResponse.blob();

// 5. Check if service is running
try {
  const health = await fetch('http://localhost:7187/health');
  const { status } = await health.json();
```

```
  console.log('Boba is running:', status === 'healthy');
} catch (e) {
  console.log('Boba not available');
}
```

12.3 Content Script Integration

For injecting search/download buttons on torrent sites:

```
// manifest.json (v3)
{
  "host_permissions": ["*://1337x.to/*", "*://thepiratebay.org/
       *"],
  "permissions": ["storage", "activeTab"],
  "content_scripts": [{
    "matches": ["*://1337x.to/*"],
    "js": ["content.js"]
  }]
}

// content.js -- Send page torrents to Boba
const torrents = extractTorrentsFromPage();
chrome.runtime.sendMessage({
  action: 'search_result',
  torrents,
  source: window.location.hostname
});
```

12.4 Authentication Handling

The extension should: 1. Detect if Boba is running (`GET /health`) 2. Prompt for qBittorrent credentials if needed (`POST /api/v1/auth/qbittorrent`) 3. Handle RuTracker CAPTCHA challenges via `GET /api/v1/auth/rutracker/captcha` 4. Store credentials securely using browser extension storage API

12.5 CORS Considerations

- By default, `ALLOWED_ORIGINS=*` in development
- For production, set `ALLOWED_ORIGINS=http://localhost:7187,chrome-extension://YOUR_EXTENSION_ID`
- The FastAPI CORS middleware handles preflight OPTIONS automatically
- Extension background scripts are not subject to CORS restrictions

12.6 Recommended Extension Architecture

```
Browser Extension (MV3)
|
+-- background.js (service worker)
|   +-- Manages SSE connections to Boba
|   +-- Caches search results
|   +-- Handles downloads
|   +-- Stores credentials securely
|
+-- popup/ (React/Vue popup)
|   +-- Search form
|   +-- Results list
|   +-- Settings (Boba URL, credentials)
|
+-- content/ (content scripts)
|   +-- Injects Boba buttons on torrent sites
|   +-- Extracts torrent info from pages
|
+-- options/ (options page)
    +-- Full settings
    +-- Tracker configuration
```

---

13. Key Files Reference

File	Purpose
`download-proxy/src/api/__init__.py`	FastAPI app, lifespan, CORS, SPA serving
`download-proxy/src/api/routes.py`	Core search/download endpoints
`download-proxy/src/api/auth.py`	Tracker auth, CAPTCHA handling
`download-proxy/src/api/streaming.py`	SSE streaming implementation
`download-proxy/src/api/hooks.py`	Hook CRUD + dispatch
`download-proxy/src/api/scheduler.py`	Scheduled search management
`download-proxy/src/merge_service/search.py`	SearchOrchestrator, data models
`download-proxy/src/merge_service/deduplicator.py`	Result deduplication

File	Purpose
`download-proxy/src/merge_service/enricher.py`	OMDB/TMDB metadata enrichment
`webui-bridge.py`	Private-tracker download bridge
`qBitTorrent-go/cmd/qbittorrent-proxy/main.go`	Go backend entry point
`qBitTorrent-go/internal/api/search.go`	Go search handlers
`qBitTorrent-go/internal/service/merge_search.go`	Go search service
`frontend/src/app/services/api.service.ts`	Angular HTTP client
`frontend/src/app/services/sse.service.ts`	Angular SSE handler
`frontend/src/app/models/search.model.ts`	TypeScript type definitions
`docker-compose.yml`	Service topology
`.env.example`	Configuration template
`docs/api/openapi.json`	Machine-readable API spec

14. Version Numbers & Compatibility

Component	Version	Notes
Python	3.12-alpine	Download proxy container
FastAPI	Latest (pip)	Merge service framework
Angular	21	Frontend SPA
Go	1.21+	Backend (Gin framework)
qBittorrent	Latest (linuxserver image)	Core torrent client
Jackett	Latest (linuxserver image)	External indexer
SQLite	3.x	Go backend only

15. Security Considerations

1. **Hardcoded credentials**: Default `admin/admin` for qBittorrent WebUI and boba-jackett
2. **Credential storage**: `.env` file is gitignored; container env vars used
3. **CORS**: `ALLOWED_ORIGINS=*` in development; restrict in production
4. **CAPTCHA TTL**: `_pending_captchas` uses TTLCache (15 min default)
5. **Path traversal**: Hook script paths validated (no `..` allowed)
6. **Credential scrubbing**: Logging uses `CredentialScrubber` filter
7. **No auth on read endpoints**: GET/HEAD pass through on boba-jackett

---

Citations

```
Claim: Boba-Base is a multi-tracker meta-search for qBittorrent
with FastAPI merge search on port 7187
Source: Boba-Base README.md
URL: https://github.com/milos85vasic/Boba-Base/blob/main/README.md
Date: 2025-04-10
Excerpt: "Multi-tracker meta-search for qBittorrent -- self-
hosted, containerised, private-tracker-aware... Merge Search
Service -- FastAPI service (:7187) that fans out across 40+
trackers"
Context: Official project documentation
Confidence: high

Claim: The data model uses Python dataclasses and Pydantic models
with no relational database
Source: Boba-Base docs/DATA_MODEL.md
URL: https://github.com/milos85vasic/Boba-Base/blob/main/docs/
DATA_MODEL.md
Date: N/A (repo file)
Excerpt: "qBittorrent-Fixed has no relational database. All
persistent state lives in qBittorrent itself and in a handful of
JSON files under config/"
Context: Architecture documentation
Confidence: high

Claim: The SSE streaming endpoint emits result_found events as
each tracker completes
Source: Boba-Base download-proxy/src/api/streaming.py
URL: https://github.com/milos85vasic/Boba-Base/blob/main/download-
proxy/src/api/streaming.py
Date: N/A (repo file)
Excerpt: "yield SSEHandler.format_event(event='result_found',
data={...})"
Context: Source code for SSE streaming
```

Confidence: high

Claim: The webui-bridge.py intercepts download requests for private trackers
Source: Boba-Base webui-bridge.py
URL: https://github.com/milos85vasic/Boba-Base/blob/main/webui-bridge.py
Date: N/A (repo file)
Excerpt: "This module solves the WebUI download issue by creating a bridge between WebUI and nova2dl.py. It intercepts download requests and handles them with proper authentication."
Context: Private tracker bridge implementation
Confidence: high

Claim: The Go backend uses admin/admin Basic Auth for mutating endpoints
Source: Boba-Base qBitTorrent-go/internal/jackettapi/auth_middleware.go
URL: https://github.com/milos85vasic/Boba-Base/blob/main/qBitTorrent-go/internal/jackettapi/auth_middleware.go
Date: N/A (repo file)
Excerpt: "adminUser/adminPass mirror the project's hardcoded WebUI credentials... Mutating requests must present these via HTTP Basic Auth"
Context: Authentication middleware source
Confidence: high

Claim: The download endpoint fetches torrents with tracker authentication and uploads to qBittorrent
Source: Boba-Base download-proxy/src/api/routes.py
URL: https://github.com/milos85vasic/Boba-Base/blob/main/download-proxy/src/api/routes.py
Date: N/A (repo file)
Excerpt: "tracker = _is_tracker_url(url); if tracker: torrent_data = await orch.fetch_torrent(tracker, url); ... session.post(f'{qbit_url}/api/v2/torrents/add', data=form)"
Context: Download handler implementation
Confidence: high

Claim: There are 48 built-in search plugins supporting both public and private trackers
Source: Boba-Base docs/PLUGINS.md
URL: https://github.com/milos85vasic/Boba-Base/blob/main/docs/PLUGINS.md
Date: N/A (repo file)
Excerpt: "This project includes multiple search engine plugins for qBittorrent... 48 plugin engines"
Context: Plugin documentation
Confidence: high

Claim: The OpenAPI spec is available at /openapi.json and frozen for CI diffing

```
Source: Boba-Base docs/api/openapi.json
URL: https://github.com/milos85vasic/Boba-Base/blob/main/docs/api/
openapi.json
Date: N/A (repo file)
Excerpt: Full OpenAPI 3.1 spec with schemas for SearchRequest,
SearchResponse, DownloadRequest, etc.
Context: Machine-readable API specification
Confidence: high

Claim: The Angular frontend uses same-origin requests with empty
baseUrl
Source: Boba-Base frontend/src/app/services/api.service.ts
URL: https://github.com/milos85vasic/Boba-Base/blob/main/frontend/
src/app/services/api.service.ts
Date: N/A (repo file)
Excerpt: "private baseUrl = '';"
Context: Frontend API service
Confidence: high
```

Research completed. This document provides a comprehensive architectural analysis of Boba-Base suitable for browser extension development planning.